Next, let's implement the `consumeOrderDispatchedMessageEvent()` method in the `OrderService` class to update the status in the database for an existing order after it's dispatched.

Listing 10.20 Implementing the logic for updating an order as dispatched

```
@Service
public class OrderService {
  ...

  public Flux<Order> consumeOrderDispatchedEvent(
    Flux<OrderDispatchedMessage> flux
  ) {
    return flux                                              ❶
      .flatMap(message ->
        orderRepository.findById(message.orderId()))         ❷
      .map(this::buildDispatchedOrder)                       ❸
      .flatMap(orderRepository::save);                       ❹
  }

  private Order buildDispatchedOrder(Order existingOrder) {
    return new Order(                                        ❺
      existingOrder.id(),
      existingOrder.bookIsbn(),
      existingOrder.bookName(),
      existingOrder.bookPrice(),
      existingOrder.quantity(),
      OrderStatus.DISPATCHED,
      existingOrder.createdDate(),
      existingOrder.lastModifiedDate(),
      existingOrder.version()
    );
  }
}
```

❶ Accepts a reactive stream of OrderDispatchedMessage objects as input

❷ For each object emitted to the stream, it reads the related order from the database.

❸ Updates the order with the "dispatched" status

❹ Saves the updated order in the database

❺ Given an order, it returns a new record with the "dispatched" status.